Classes e Herança em Python

Estruturas de Dados



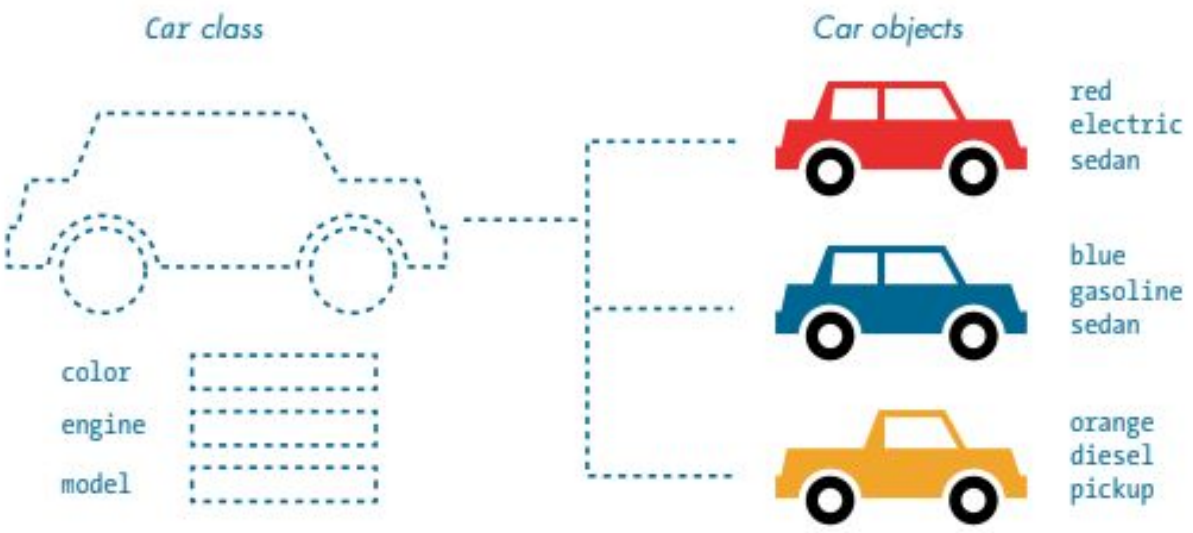
Universidade de Brasília
Departamento de Ciência da Computação

O que é Programação Orientada a Objetos?

- Programação Orientada a Objetos é um paradigma que organiza o código em torno de "objetos" e "classes".

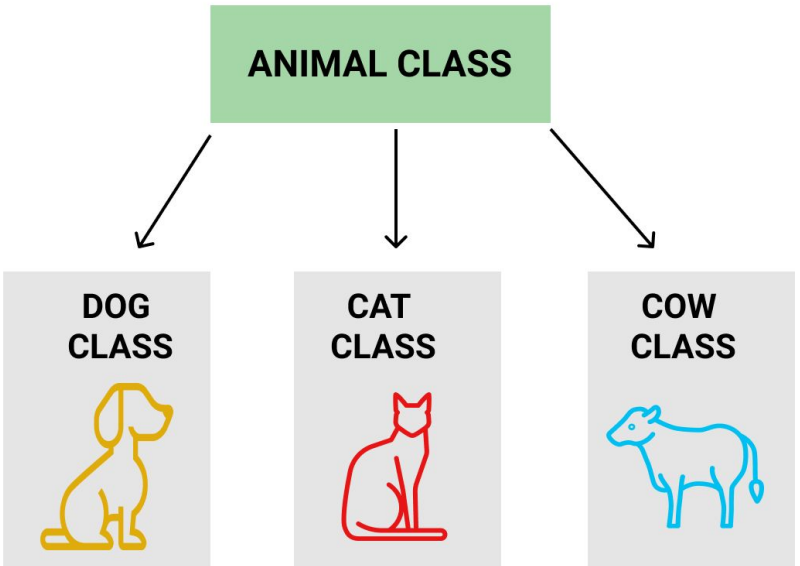
O que é Programação Orientada a Objetos?

- Programação Orientada a Objetos é um paradigma que organiza o código em torno de "objetos" e "classes".



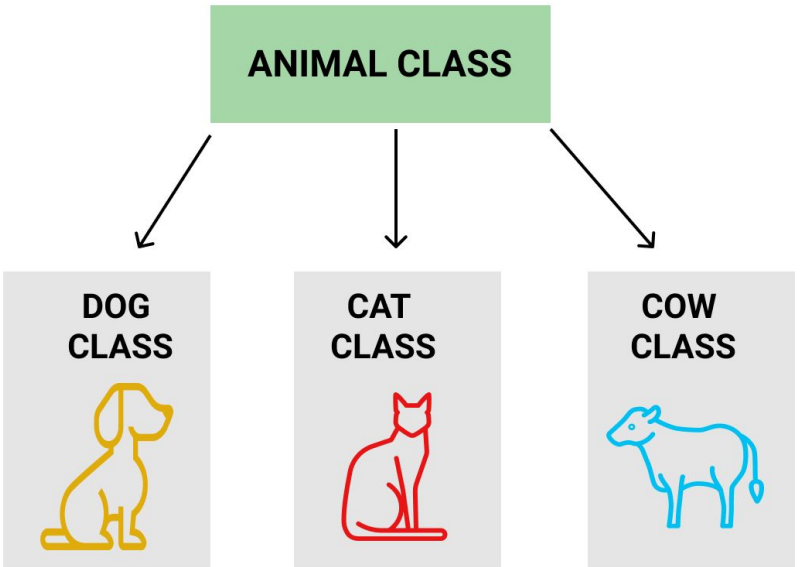
O que é Programação Orientada a Objetos?

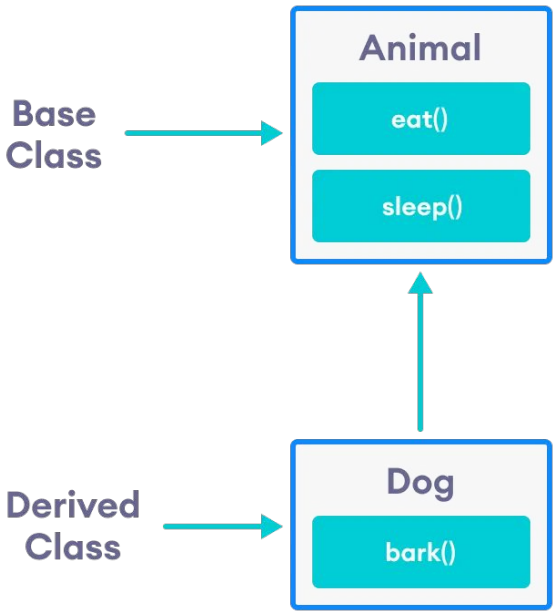
- Programação Orientada a Objetos é um paradigma que organiza o código em torno de "objetos" e "classes".



O que é Programação Orientada a Objetos?

- Programação Orientada a Objetos é um paradigma que organiza o código em torno de "objetos" e "classes".





O que é Programação Orientada a Objetos?

- Programação Orientada a Objetos é um paradigma que organiza o código em torno de "objetos" e "classes".
- Pilares principais:
  - Encapsulamento
  - Herança
  - Polimorfismo

O que é Programação Orientada a Objetos?

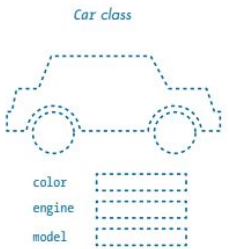
- **Encapsulamento:** Técnica de ocultar os detalhes internos de uma classe, expondo apenas o que é necessário. Permite proteger dados e limitar o acesso direto a certos atributos
- **Herança:** Mecanismo que permite criar uma nova classe (subclasse) baseada em uma classe existente (superclasse), herdando seus atributos e métodos.
- **Polimorfismo:** Capacidade de diferentes classes usarem métodos com o mesmo nome, mas com comportamentos distintos.

Classes e Objetos

- **Classe:** Modelo ou molde de criação de objetos.
- **Objeto:** Instância de uma classe com dados e métodos específicos.

Classes e Objetos

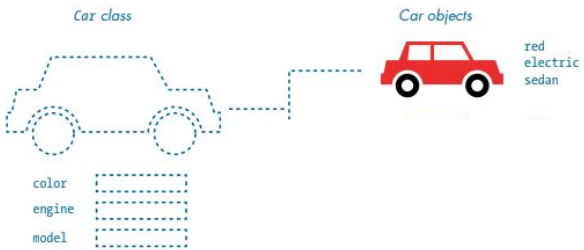
- **Classe:** Modelo ou molde de criação de objetos.
- **Objeto:** Instância de uma classe com dados e métodos específicos.




```
class Carro:
    pass
```

Classes e Objetos

- **Classe:** Modelo ou molde de criação de objetos.
- **Objeto:** Instância de uma classe com dados e métodos específicos.




```
class Carro:
    pass
```

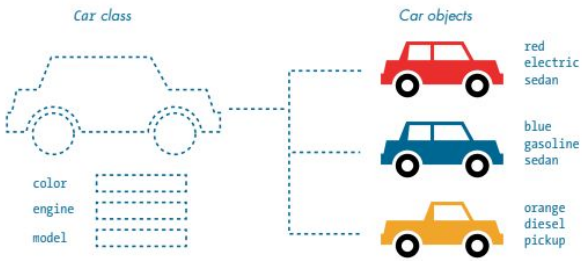
classe

```
meu_carro = Carro()
```

objeto

Classes e Objetos

- **Classe:** Modelo ou molde de criação de objetos.
- **Objeto:** Instância de uma classe com dados e métodos específicos.




```
class Carro:
    pass
```
classe

```
meu_carro = Carro()
```
objeto

```
outro_carro = Carro()
```
outro objeto

```
o_carro = Carro()
```
outro objeto

Estrutura de uma classe

Uma classe contém atributos (dados) e métodos (funções).

Sintaxe:

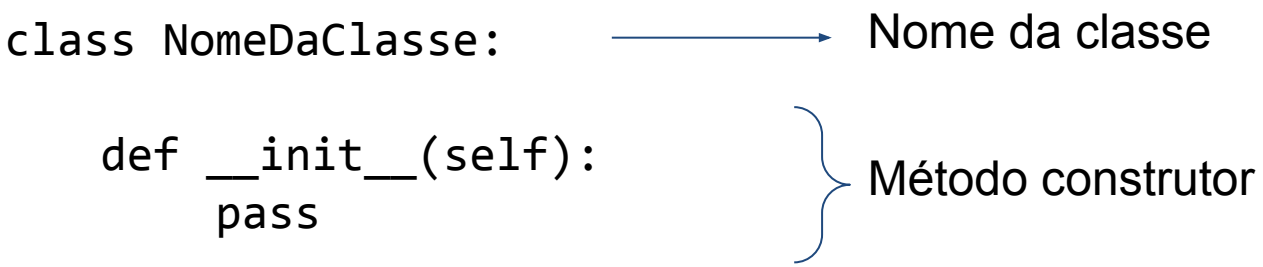
```
class NomeDaClasse:

    def __init__(self):
        pass
```

Estrutura de uma classe

Uma classe contém atributos (dados) e métodos (funções).

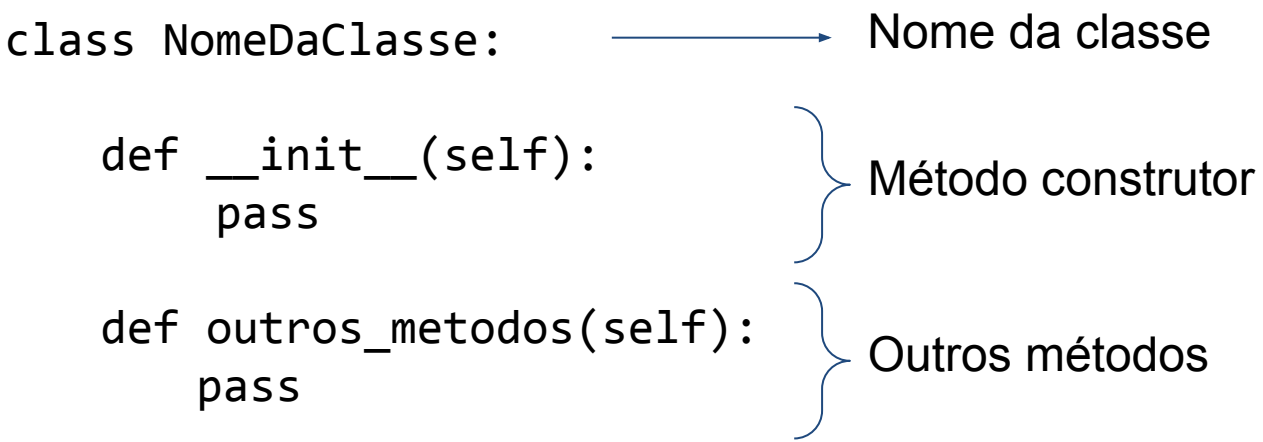
Sintaxe:



Estrutura de uma classe

Uma classe contém atributos (dados) e métodos (funções).

Sintaxe:



Método Construtor __init__

- Método especial do python conhecido como construtor.
- Ele é executado automaticamente sempre que um novo objeto de uma classe é criado.
- Inicializa os atributos de uma nova instância da classe.
- Define o estado inicial do objeto com base nos parâmetros passados ao criar o objeto.

Entendendo o `self` no Método `__init__`

- `self` é uma referência à instância atual da classe.
- Ele permite acessar os atributos e métodos da própria instância dentro da classe.
- No método `__init__`, usamos `self` para inicializar os atributos de um objeto específico.
- Cada vez que um objeto é criado, `self` garante que os atributos sejam únicos para aquela instância.

Criando uma Classe

Criar uma classe Carro com atributos básicos.

```
class Carro:
    def __init__(self, marca, ano):
        self.marca = marca
        self.ano = ano
```

Instanciando um objeto

Criar um ou mais objetos de uma classe.

```
c1 = Carro("Toyota", 2020)
c2 = Carro("Fiat", 1990)
```

Getters e Setters

São métodos que controlam o acesso aos atributos.

```
class Carro:
    def __init__(self, marca, ano):
        self._marca = marca
        self.ano = ano
        self.km = 0

    def get_marca(self):
        return self._marca

    def set_km(self, km):
        self.km = km
```

Métodos de Classe

Manipulam atributos e implementam ações.

```
class Carro:
    ...
    def exibir_detalhes(self):
        print(f"Marca: {self.marca}")
        print(f"Ano: {self.ano}")
        print(f"Kms: {self.km}")
```

Métodos especiais ou mágicos

`__str__`: Retorna uma string para representar o objeto.
`__repr_`: Retorna uma string para debug.

```
class Carro:
    ...
    def __str__(self):
        return f"{self.marca} - {self.ano}"
```

Conceito de Herança

Permite que uma classe herde métodos e atributos de outra classe.

Conceito de Herança

Permite que uma classe herde métodos e atributos de outra classe.

```
class Veiculo:
    def __init__(self, marca, ano):
        self.marca = marca
        self.ano = ano

    def exibir_detalhes(self):
        print(f"Carro: {self.marca} - {self.ano}")
```

Conceito de Herança

Permite que uma classe herde métodos e atributos de outra classe.

```
class Carro(Veiculo):
    def __init__(self, marca, ano, modelo):
        super().__init__(marca, ano)
        self.modelo = modelo
```

Conceito de Herança

Permite que uma classe herde métodos e atributos de outra classe.

```
c1 = Veiculo('Caloi', 2020)
c1.exibir_detalhes()

c2 = Carro('Toyota', 2023, 'Corolla')
c2.exibir_detalhes()
```

Polimorfismo

Permite que diferentes classes tenham métodos com o mesmo nome.

Exemplo: Gato e Cachorro com método emitir_som()

Polimorfismo

Permite que diferentes classes tenham métodos com o mesmo nome.

Exemplo: Gato e Cachorro com método emitir_som()

```
class Animal:
    def emitir_som(self):
        pass
```

Polimorfismo

Permite que diferentes classes tenham métodos com o mesmo nome.

Exemplo: Gato e Cachorro com método emitir_som()

```
class Animal:
    def emitir_som(self):
        pass

class Gato(Animal):
    def emitir_som(self):
        print("Miau")
```

Dúvidas e Perguntas!